



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

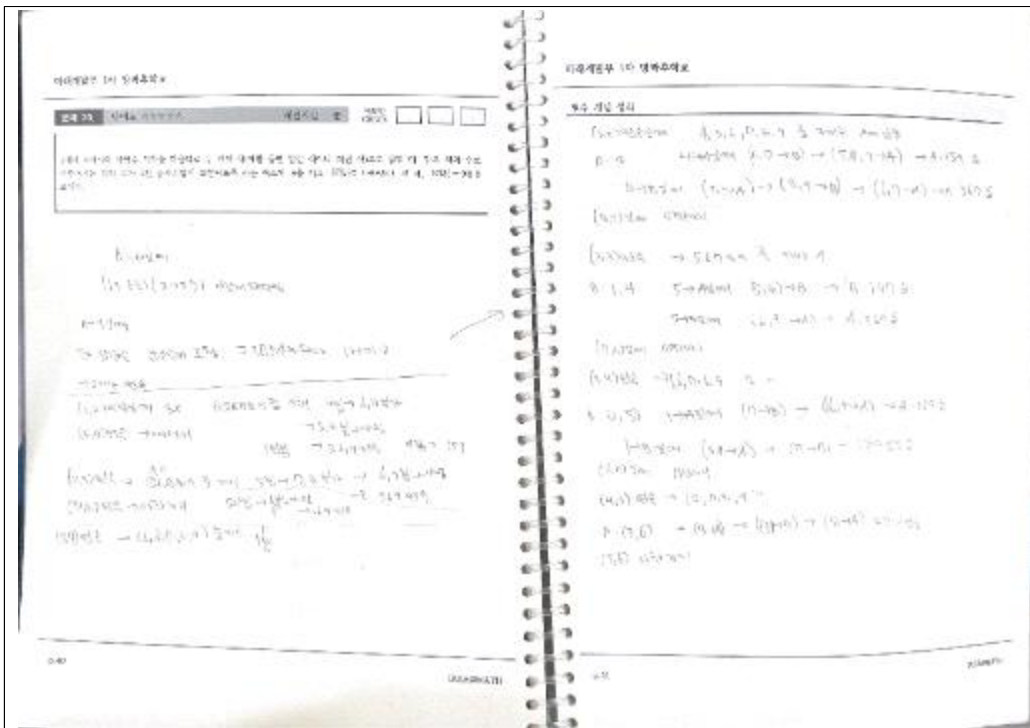
- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

1에서 n까지의 자연수 각각을 마음대로 두 가지 색으로 칠할 때 같은 색의 수로 이루어지는 항의 수가 k인 등차수열이 포함되도록 하는 최소의 n을 $W(k)$ 라고 나타낸다. 이 때 $W(3) = 9$ 임을 보이라

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

공의 최소의 수를 구했다고 해도 그때 조건을 만족하는지 확인하기 어렵고 그 전에는 확실히 조건을 만족하지 못하는 것인지 확인하기 어려워 시뮬레이션을 통해 시각적으로 확인시켜주고 싶었다.

4. 사용한 AI 플랫폼을 모두 적으세요.

gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

1에서 n 까지의 자연수 각각을 마음대로 두 가지 색으로 칠할 때 같은 색의 수로 이루어지는 항의 수가 k 인 등차수열이 포함되도록 하는 최소의 n 을 $W(k)$ 라고 나타낸다. 이 때 $W(3) = 9$ 임을 보여라
이 문제를 시뮬레이션을 이용해 이해를 돕게 하고 싶어. html로 $W(k)$ 를 구하는 과정을 시뮬레이션 해줘. k 값을 조절 할 수 있고 실제 공들을 빨, 파 색으로 칠해가며 조건을 만족하면 1더하고 이런 식으로

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>W(k) 시뮬레이션: 반 데르 바르덴 수</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; line-height: 1.6; padding: 20px; background-color: #f4f7f6; }
    .container { max-width: 800px; margin: auto; background: white; padding: 30px; border-radius: 12px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }
    h2 { color: #2c3e50; border-bottom: 2px solid #eee; padding-bottom: 10px; }
    .controls { margin-bottom: 20px; display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
    .ball-container { display: flex; gap: 8px; margin: 20px 0; flex-wrap: wrap; min-height: 60px; align-items: center; }
    .ball { width: 40px; height: 40px; border-radius: 50%; display: flex; align-items: center; justify-content: center; color: white; font-weight: bold; cursor: pointer; transition: transform 0.2s; user-select: none; border: 2px solid #ddd; }
    .ball:hover { transform: scale(1.1); }
    .ball.red { background-color: #e74c3c; border-color: #c0392b; }
    .ball.blue { background-color: #3498db; border-color: #2980b9; }
    .ball.highlight { ring: 4px solid #f1c40f; box-shadow: 0 0 15px #f1c40f; transform: scale(1.2); border-color: #f1c40f; }
    .info { background: #e8f4fd; padding: 15px; border-left: 5px solid #3498db; margin: 10px 0; }
```

```

    button { padding: 8px 16px; border: none; border-radius: 4px; cursor: pointer; background:
#2c3e50; color: white; font-weight: bold; }
    button:hover { background: #34495e; }
    .result-box { font-weight: bold; margin-top: 10px; min-height: 24px; }
    input { padding: 6px; width: 50px; border: 1px solid #ddd; border-radius: 4px; }
</style>
</head>
<body>

<div class="container">
    <h2>$W(k)$ 시뮬레이터</h2>
    <p>1부터 $n$까지의 자연수를 두 가지 색(빨강/파랑)으로 칠할 때, 길이 $k$인 <b>단색 등차수열</b>이 존
재하는지 확인합니다.</p>

    <div class="controls">
        <label>길이 $k$: </label>
        <input type="number" id="kVal" value="3" min="2" max="5">
        <label>숫자 개수 $n$: </label>
        <input type="number" id="nVal" value="9" min="1" max="40">
        <button onclick="resetApp()">초기화 및 적용</button>
        <button onclick="checkAP()" style="background: #27ae60;">등차수열 찾기</button>
        <button onclick="findCounterExample()" style="background: #8e44ad;">반례 찾기(자동)</button>
    </div>

    <div class="info" id="status">
        공을 클릭하여 색을 바꿔보세요! (빨강 ↔ 파랑)
    </div>

    <div class="ball-container" id="ballStage"></div>

    <div class="result-box" id="result"></div>

    <hr>
    <h3>$W(3) = 9$의 의미</h3>
    <ul>
        <li><b>$n=8$일 때:</b> <code style="color: #e74c3c">R</code> <code style="color:
#e74c3c">R</code> <code style="color: #3498db">B</code> <code style="color: #3498db">B</code> <code
style="color: #e74c3c">R</code> <code style="color: #e74c3c">R</code> <code style="color:
#3498db">B</code> <code style="color: #3498db">B</code> 처럼 칠하면 길이 3인 단색 등차수열이 없습니
다.</li>
        <li><b>$n=9$일 때:</b> 어떤 조합으로 칠해도(총 $2^9=512$가지) 반드시 하나 이상의 단색 등차수열이
발견됩니다.</li>
    </ul>
</div>

<script>
    let n = 9;
    let k = 3;

```

```
let colors = []; // 0: red, 1: blue
```

```
function init() {  
  n = parseInt(document.getElementById('nVal').value);  
  k = parseInt(document.getElementById('kVal').value);  
  colors = new Array(n).fill(0);  
  render();  
  document.getElementById('result').innerText = "";  
}
```

```
function render() {  
  const stage = document.getElementById('ballStage');  
  stage.innerHTML = "";  
  colors.forEach((c, i) => {  
    const ball = document.createElement('div');  
    ball.className = `ball ${c === 0 ? 'red' : 'blue'}`;  
    ball.innerText = i + 1;  
    ball.onclick = () => {  
      colors[i] = 1 - colors[i];  
      render();  
      document.getElementById('result').innerText = "";  
    };  
    stage.appendChild(ball);  
  });  
}
```

```
function checkAP() {  
  const balls = document.querySelectorAll('.ball');  
  balls.forEach(b => b.classList.remove('highlight'));  
  
  let found = false;  
  // 등차수열 검사: a (시작점), d (공차)  
  //  $a + (k-1)d < n$   
  for (let a = 0; a < n; a++) {  
    for (let d = 1; a + (k - 1) * d < n; d++) {  
      let sequence = [];  
      let color = colors[a];  
      let isMatch = true;  
  
      for (let j = 0; j < k; j++) {  
        let idx = a + j * d;  
        if (colors[idx] !== color) {  
          isMatch = false;  
          break;  
        }  
        sequence.push(idx);  
      }  
    }  
  }
```

```

        if (isMatch) {
            sequence.forEach(idx => balls[idx].classList.add('highlight'));
            document.getElementById('result').innerText =
                `발견! 시작점: ${a+1}, 공차: ${d}, 색: ${color === 0 ? '빨강' : '파랑'}`;
            found = true;
            return; // 하나만 찾아도 종료
        }
    }
}

if (!found) {
    document.getElementById('result').innerText = "단색 등차수열이 없습니다.";
}
}

```

```
function findCounterExample() {
```

```
    // 단순 브루트포스로 반례 탐색 (n이 작을 때만)
```

```
    if (n > 25 && k < 3) { alert("n이 너무 커서 오래 걸릴 수 있습니다."); }
```

```
    let foundCE = false;
```

```
    const total = Math.pow(2, n);
```

```
    // n이 작으므로 비트 연산으로 모든 경우의 수 탐색 가능 (최대 2^20 정도)
```

```
    for (let i = 0; i < Math.min(total, 100000); i++) {
```

```
        let tempColors = i.toString(2).padStart(n, '0').split('').map(Number);
```

```
        if (!hasAP(tempColors, k)) {
```

```
            colors = tempColors;
```

```
            render();
```

```
            document.getElementById('result').innerText = "반례를 찾았습니다!";
```

```
            foundCE = true;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (!foundCE) {
```

```
        document.getElementById('result').innerText = `n=${n}에서 길이 ${k}인 단색 등차수열이 없는 배  
치를 찾을 수 없습니다. (W(${k}) ≤ ${n}일 가능성 높음)`;
```

```
    }
```

```
}
```

```
function hasAP(arr, kLen) {
```

```
    let len = arr.length;
```

```
    for (let a = 0; a < len; a++) {
```

```
        for (let d = 1; a + (kLen - 1) * d < len; d++) {
```

```
            let match = true;
```

```
            for (let j = 1; j < kLen; j++) {
```

```
                if (arr[a] !== arr[a + j * d]) {
```

```
                    match = false;
```

```
                    break;
```

```
        }  
    }  
    if (match) return true;  
}  
}  
return false;  
}  
  
function resetApp() {  
    init();  
}  
  
init();  
</script>  
  
</body>  
</html>
```

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

숫자의 개수를 직접 입력한다는 점에서 그 답이 왜 나오냐가 아니라 그 답이 맞느냐만 확인시켜준다. 이 부분을 고쳐야 한다. 또한 반례를 바로 보여줘서 과정을 알기 어렵다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째: 우선 k만 입력하게 해줘. k를 넣으면 $W(k)$ 를 구하게. 그리고 공을 k개부터 시작해서 모든 조합을 해보면서 조건을 만족하는지 시각적으로 확인시켜주고 조건 만족하는게 없을 때 공을 하나씩 늘리고 이런식으로 해줘.
3번째: 일단 속도가 너무 빨라. 최대 속도는 이대로 하고 최소 속도를 더 느리게 해줘. 그리고 이미 앞쪽에 조건을 만족하는게 있으면 뒤는 볼 필요 없잖아. 예를 들어 k=5를 했는데 이미 앞쪽에 5개 연속으로 나왔는데 뒤의 공들을 막 돌리는건 시간낭비야. 이 시간 낭비를 없애주고 관련되어 설명도 작성해줘
4번째: 최대 지연 시간을 500ms로 해. 그리고 공 하나를 추가할 때 너무 오래 기다려. 그때는 지연시간의 세 배만큼만 기다리게 해줘

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

파일로 첨부(제목: 최종 시뮬레이터 모습1)

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

k값과 시뮬레이션 속도를 입력하면 조건을 만족하는 최소 공의 개수를 하나하나 조합을 보여주면서 찾아준다. 앞쪽에서 이미 등차수열을 이뤄서 뒤 공의 색을 바꾸는 것이 의미 없는 경우에는 생략한다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

답을 확실하게 해주는데 도움이 되었다. 왜 W(3)이 9인지 이해하기 쉬워졌다. 하지만 W(4)부터 값이 기하급수적으로 커져 시뮬레이션이 오래 걸렸다. 특히 W(5)부터는 시뮬레이션을 돌리는 의미가 없어져서 이 부분이 아쉬웠다.

[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

- $f(x) = (x^2+3x+a)(x+1)^2$
- (1) $f(x)$ 의 최솟값이 음수인 a 범위 구해라
 - (2) $a < 2$ 일 때 $f(x)$ 는 a_1, a_2 ($a_1 < a_2$)에서 극소를 가진다. $f(a_1) < f(a_2)$ 임을 보여라
 - (3) $f(x)$ 가 $x < b$ 에서 단조감소하고 $x = b$ 에서 최솟값을 가질 때 a 의 범위를 구해라

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

그래프의 극대, 극소, 최솟값에 대해 탐구하는 문제이므로 그래프를 직접 그려 해당 조건에서 문제의 조건이 만족함을 시각적으로 보여주면 이해하기 더 쉬울거라 생각했기 때문이다.

4. 사용한 AI 플랫폼을 모두 적으세요.

gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

함수 $f(x) = (x^2 + 3x + a)(x + 1)^2$ 에 대하여 아래 물음에 답하십시오.

(1) f 의 최솟값이 음이 되기 위한 실수 a 의 범위를 구하십시오.

(2) $a < 2$ 일 때 f 는 두 개의 극솟값을 가진다. f 가 극솟값을 가지는 x 의 값을 a_1, a_2 ($a_1 < a_2$) 라 할 때 $f(a_1) < f(a_2)$ 임을 보여라.

(3) f 가 $x < b$ 에서 단조감소하고 $x = b$ 에서 최솟값을 가진다고 하자. 이 때 a 의 범위를 구하십시오

이 문제를 푸는데 도움이 되는 시뮬레이터를 html로 만들거야. a에 따른 함수 f를 그려줘. a의 값은 -10 부터 10사이에서 정할수도 있고 자동으로 움직이게 할 수도 있어. 1번 문제에서는 f의 최솟값을 찍어주고 2번에서는 a의 범위를 해당하는 범위로 줄이고 a_1, a_2 를 찍어줘. 3번에서는 b 를 찍어주고 a 값을 변화시켜

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>함수 f(x) 시뮬레이터</title>
  <script src="https://cdn.plot.ly/plotly-2.27.0.min.js"></script>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; max-width: 900px;
margin: 0 auto; padding: 20px; background-color: #f8f9fa; }
    .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 4px 6px
rgba(0,0,0,0.1); margin-bottom: 20px; }
    .controls { display: flex; flex-direction: column; gap: 15px; }
    .tabs { display: flex; gap: 10px; margin-bottom: 10px; }
    button { padding: 8px 16px; cursor: pointer; border: 1px solid #007bff; background: white;
color: #007bff; border-radius: 4px; font-weight: bold; transition: 0.2s; }
    button.active { background: #007bff; color: white; }
    button:hover { background: #e9ecef; color: #0056b3; }
    button.active:hover { background: #0056b3; color: white; }
    .slider-container { display: flex; align-items: center; gap: 15px; }
    input[type="range"] { flex-grow: 1; }
    #plot { width: 100%; height: 500px; }
    .info-panel { padding: 15px; background: #e9ecef; border-radius: 4px; font-size: 15px; }
  </style>
</head>
<body>

<div class="card">
  <h2>함수 f(x) = (x2 + 3x + a)(x + 1)2 시뮬레이터</h2>
```

```
<div class="tabs">
  <button id="btn1" class="active" onclick="setMode(1)">문제 1: 최솟값 범위</button>
  <button id="btn2" onclick="setMode(2)">문제 2: a < 2 극솟값 비교</button>
  <button id="btn3" onclick="setMode(3)">문제 3: 단조감소 및 최솟값 b</button>
</div>
```

```
<div class="controls">
  <div class="slider-container">
    <label for="a-slider"><strong>a 값:</strong></label>
    <input type="range" id="a-slider" min="-10" max="10" step="0.01" value="0">
    <span id="a-val" style="width: 50px; font-weight: bold;">0.00</span>
    <button id="play-btn" onclick="togglePlay()">▶ 자동 재생</button>
  </div>
  <div id="info" class="info-panel">여기에 상태 정보가 표시됩니다.</div>
</div>
</div>
```

```
<div class="card">
  <div id="plot"></div>
</div>
```

```
<script>
  let mode = 1;
  let aValue = 0;
  let playInterval = null;
  let isPlaying = false;

  // 함수 f(x)
  function f(x, a) {
    return (x * x + 3 * x + a) * Math.pow(x + 1, 2);
  }

  // 도함수 g(x) = 4x^2 + 11x + 2a + 3 의 근 찾기
  function getGRoots(a) {
    const D = 121 - 16 * (2 * a + 3);
    if (D < 0) return [];
    if (D === 0) return [-11 / 8];
    return [
      (-11 - Math.sqrt(D)) / 8,
      (-11 + Math.sqrt(D)) / 8
    ].sort((a, b) => a - b);
  }

  function setMode(newMode) {
    mode = newMode;
    document.getElementById('btn1').classList.remove('active');
    document.getElementById('btn2').classList.remove('active');
```

```
document.getElementById('btn3').classList.remove('active');
document.getElementById('btn' + mode).classList.add('active');
```

```
const slider = document.getElementById('a-slider');
```

```
if (mode === 2) {
    slider.max = "1.99";
    if (aValue >= 2) {
        aValue = 1.99;
        slider.value = aValue;
    }
} else {
    slider.max = "10";
}
```

```
updatePlot();
```

```
}
```

```
function togglePlay() {
```

```
    const btn = document.getElementById('play-btn');
```

```
    if (isPlaying) {
```

```
        clearInterval(playInterval);
```

```
        btn.innerText = "▶ 자동 재생";
```

```
        isPlaying = false;
```

```
    } else {
```

```
        btn.innerText = "⏏ 정지";
```

```
        isPlaying = true;
```

```
        playInterval = setInterval(() => {
```

```
            const slider = document.getElementById('a-slider');
```

```
            let nextA = aValue + 0.05;
```

```
            if (nextA > parseFloat(slider.max)) nextA = parseFloat(slider.min);
```

```
            slider.value = nextA;
```

```
            aValue = nextA;
```

```
            updatePlot();
```

```
        }, 50);
```

```
    }
```

```
}
```

```
document.getElementById('a-slider').addEventListener('input', function(e) {
```

```
    aValue = parseFloat(e.target.value);
```

```
    updatePlot();
```

```
});
```

```
function updatePlot() {
```

```
    document.getElementById('a-val').innerText = aValue.toFixed(2);
```

```
    const xData = [];
```

```
    const yData = [];
```

```
    for (let x = -4.5; x <= 2.5; x += 0.02) {
```

```
        xData.push(x);
```

```
        yData.push(f(x, aValue));
```

```

}

const trace1 = {
  x: xData,
  y: yData,
  type: 'scatter',
  mode: 'lines',
  name: 'f(x)',
  line: { color: '#007bff', width: 2 }
};

const layout = {
  margin: { t: 30, b: 40, l: 50, r: 20 },
  xaxis: { title: 'x', range: [-4, 2], zeroline: true, zerolinecolor: '#999' },
  yaxis: { title: 'f(x)', range: [-10, 20], zeroline: true, zerolinecolor: '#999' },
  showlegend: false,
  annotations: [],
  shapes: []
};

let infoText = '';
const traces = [trace1];

if (mode === 1) {
  // 최솟값 추적
  let minX = -1;
  let minY = 0;
  if (aValue < 2.25) {
    const roots = getGRoots(aValue);
    if (roots.length > 0) {
      minX = roots[0];
      minY = f(minX, aValue);
    }
  }
  traces.push({
    x: [minX], y: [minY],
    mode: 'markers+text',
    marker: { color: 'red', size: 10 },
    text: ['최솟값'], textposition: 'bottom center'
  });

  infoText = `현재 최솟값: ${minY.toFixed(3)} (x = ${minX.toFixed(3)})<br>`;
  if (minY < 0) infoText += `<span style="color:red; font-weight:bold;">최솟값이 음수입니다! (조건 a < 2.25 만족)</span>`;
  else infoText += `최솟값이 0 이상입니다.`;
}
else if (mode === 2) {
  // 극솟값 비교 (a < 2)

```

```

const roots = getGRoots(aValue);
if (roots.length === 2 && aValue < 2) {
    const a1 = roots[0];
    const a2 = roots[1];
    traces.push({
        x: [a1, a2], y: [f(a1, aValue), f(a2, aValue)],
        mode: 'markers+text',
        marker: { color: ['purple', 'green'], size: 10 },
        text: ['a1', 'a2'], textposition: 'bottom center'
    });
    infoText = `a1 = ${a1.toFixed(3)}, f(a1) = ${f(a1, aValue).toFixed(3)}<br>
        a2 = ${a2.toFixed(3)}, f(a2) = ${f(a2, aValue).toFixed(3)}<br>
        <strong>확인: f(a1)  ${f(a1, aValue)} < f(a2, aValue) ?  '<'  :  '>='}
f(a2)</strong>`;
    } else {
        infoText = `a < 2 일 때만 두 개의 극솟값이 존재합니다.`;
    }
}
else if (mode === 3) {
    // 단조감소 및 최솟값 조건
    let b = null;
    let isSatisfied = false;

    if (Math.abs(aValue - 2) < 0.02) {
        b = -1.75;
        isSatisfied = true;
    } else if (aValue >= 73/32) {
        b = -1;
        isSatisfied = true;
    }

    if (isSatisfied && b !== null) {
        traces.push({
            x: [b], y: [f(b, aValue)],
            mode: 'markers+text',
            marker: { color: 'orange', size: 12 },
            text: ['b'], textposition: 'top center'
        });

        // x < b 구간 하이라이트
        layout.shapes.push({
            type: 'rect',
            xref: 'x', yref: 'paper',
            x0: -4, y0: 0,
            x1: b, y1: 1,
            fillcolor: 'rgba(255, 165, 0, 0.2)',
            line: { width: 0 }
        });
    }
}

```

```

        infoText = `<span style="color:green; font-weight:bold;">조건 만족!</span> x <
        ${b.toFixed(3)} 에서 단조감소하며 x = ${b.toFixed(3)} 에서 최솟값을 가집니다.`;
    } else {
        infoText = `<span style="color:red;">조건 불만족.</span> (조건: a = 2 또는 a >= 2.28125)`;
    }
}

document.getElementById('info').innerHTML = infoText;
Plotly.react('plot', traces, layout, {displayModeBar: false});
}

// 초기화
updatePlot();
</script>

</body>
</html>

```

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

가시성이 부족하다. 임계점이 훅 지나가서 어디가 임계점인지 알기 힘들다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째: 그래프 보이는 부분이 너무 작아서 함수의 일부분만 보여. 확대 축소를 할 수 있게 해줘. 또한 모든 상황에서 절편과 극값들은 표시를 해줘. 그리고 해당하는 문제에서 중요시해야할 부분 (예를 들어 2번에선 두 극소점)은 더 강조를 해줘. 그리고 a값 변하는 슬라이드 이동할 때 1번, 3번에선 임계점에서 잠시 멈추게 해줘 (1번은 9/4가 임계점 3번은 73/32와 9/4가 임계점) 그리고 임계점에 도달했을 때 답을 알려줘. 소수로 말고 분수로

3번째: a값이 변할때마다 확대비율이 초기화돼서 사실상 확대 축소를 못해. 이점 고쳐줘

4번째: 2번에 두 극솟값 비교를 위해 각 극솟점에서 x축과 평행한 직선을 그려줘. 그리고 3번이 틀렸어. 일단 a=2 일때는 임계점이 아니고 $a < 9/4$ 또는 $a >= 73/32$ 인 모든 경우가 조건을 만족하잖아. 이 점 고려해서 다시 작성해 줘

5번째: 3번 문제를 좀 읽어봐 $a <= 9/4$ 이면 극솟값중 왼쪽이 최소잖아. 그러면 그 왼쪽은 단조감소하잖아

6번째: 3번에서 두 임계점 사이 거리가 너무 짧아서 조건을 만족하지 않는 a를 거의 못보고 지나쳐. 두 임계점 사이에서 a가 엄청 조금씩 움직이게 해줘.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

파일 따로 첨부(제목: 최종 시뮬레이션 모습2)

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

문제1, 2, 3버튼을 눌러 각 문제의 상황의 그래프를 볼 수 있다.
a값을 슬라이드로 조절해 그래프의 개형이 어떻게 변하는지 확인 할 수 있다.
a값을 자동으로 조절 시킬 수 있다.
a가 임계점일 때 잠시 멈춘다

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

1번 문제에서 그래프의 실제 최솟값이 음수임을 시각적으로 확인 할 수 있었다.
2번 문제에서 $f(a1)$ 이 언제나 더 작음을 시각적으로 확인 할 수 있었다.
3번 문제에서 $f(x)$ 가 해당 범위에서 단조 감소함을 시각적으로 확인 할 수 있었다.
도움이 되지 않은 부분은 이 그래프는 단순히 시각적으로 그래프가 조건을 만족해 보인다면 확인 할 수 있고 수식적으로 정확히 만족하는지는 확인시켜주지 못한다.

1. [마무리]

2. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

생성형 ai는 수학 문제를 시각적으로 표현하는데 도움을 준다. 또한 계산 실수를 하지 않는다는 점, 복잡한 수식도 풀 수 있다는 점에서 가치가 있다. 하지만 복잡하거나 이해하기 어려운 문제들은 여전히 풀지 못한다. 그런 경우에 인간이 방법만 알려줘도 ai는 그 문제를 잘 풀 수 있다. 이런 면에서 ai가 발전하더라도 수학적인 전문 지식을 가취야한다고 할 수 있다. 또한 ai 자체가 통계적인 확률로 행동하는 만큼 ai를 더 잘 이해하고 잘 사용하려면 수학적 지식을 갖추는 것이 필요 불가결하다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

나는 경제, 물리 현상, 디자인 등을 수학적으로 분석하고 구현해 보고 싶다. 따라서 AI를 활용해 수학 문제를 시뮬레이터로 만든 경험은 데이터와 현상을 시각화하는 연습처럼 느껴졌다. 또한 어떤 문제를 맞닥뜨렸을 때 이를 그래프나 움직임으로 표현하면 훨씬 쉽게 이해할 수 있고, 복잡한 문제를 단순한 요소로 나누는 과정이 중요하다는 것을 깨달았다. 이러한 점에서 이번 경험은 내가 희망하는 진로 분야에서 실무적인 문제를 해결할 때, 단순히 공식을 아는 것보다 현상을 모델링하고 시각적으로 검증하는 능력이 중요하다는 시사점을 주었다. 실제 문제는 여러 변수가 얹혀 있기 때문에, AI를 활용해 조건을 바꾸어 가며 시뮬레이션하고 결과를 비교하는 과정이 필요하다. 앞으로도 AI를 답을 대신 내는 도구가 아니라, 복잡한 현상을 분석하고 더 합리적인 해결책을 찾는 협업 도구로 활용하고 싶다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

ai가 정말 유용하다는 것을 느꼈다. 매우 긴 코드를 단순한 프롬프트만으로 몇 초만에 작성한다는 것이 신기했다. 또한 ai를 사용하더라도 관련 전문 지식이 필요하다고 느꼈다. 프롬프트를 작성할 때 그 문제에 대해 더 잘 이해한 상태에서 작성할수록 결과가 더 좋았기 때문이다.